

Drone Controller

Ashmit Dutta, Anmol Shah

May 12, 2026

ECE 395: Advanced Digital Projects Laboratory
University of Illinois at Urbana-Champaign

Abstract

We designed and built a handheld controller PCB that pairs with a quadcopter to provide wireless flight control. The controller is centered on an STM32L031K6T6 microcontroller that samples two analog COM-09032 thumb-joysticks via its on-chip ADC, reads orientation data from a Bosch BMI270 IMU over I²C, and transmits information to the drone over a sub-GHz link using a TI CC1101 RF transceiver. The transceiver was originally designed around the 433 MHz but switched to 315 MHz to better match the circuit components and antenna we had on hand. During our demo, our PCB had short circuited and we were not able to fully show the work we did over this semester. This report covers the system architecture, hardware design, firmware design, lessons learned from a number of build issues we encountered, and recommendations for future ECE 395 teams attempting a similar controller project.

Introduction	2
Overview	2
Understanding the Drone	2
System Block Diagram	3
System Overview	5
Board Operation	5
Hardware Design	6
Overview	6
Power/GND	8
Microcontroller (STM32L031K6T6)	9
Communications	11
Joystick Input (COM-09032 × 2)	13
Gyroscope / IMU Input (BMI270)	13
Headers and Connectors	14
Firmware Design	16
Overview	16
STM Configuration	16
Flowchart	18
Initialization	18
Main Loop	19
Helper Functions	19
Conclusion	21
Issues	21
What could be done in the future?	21
Tips for future projects regarding drone controllers	21
Final Remarks	22

Introduction

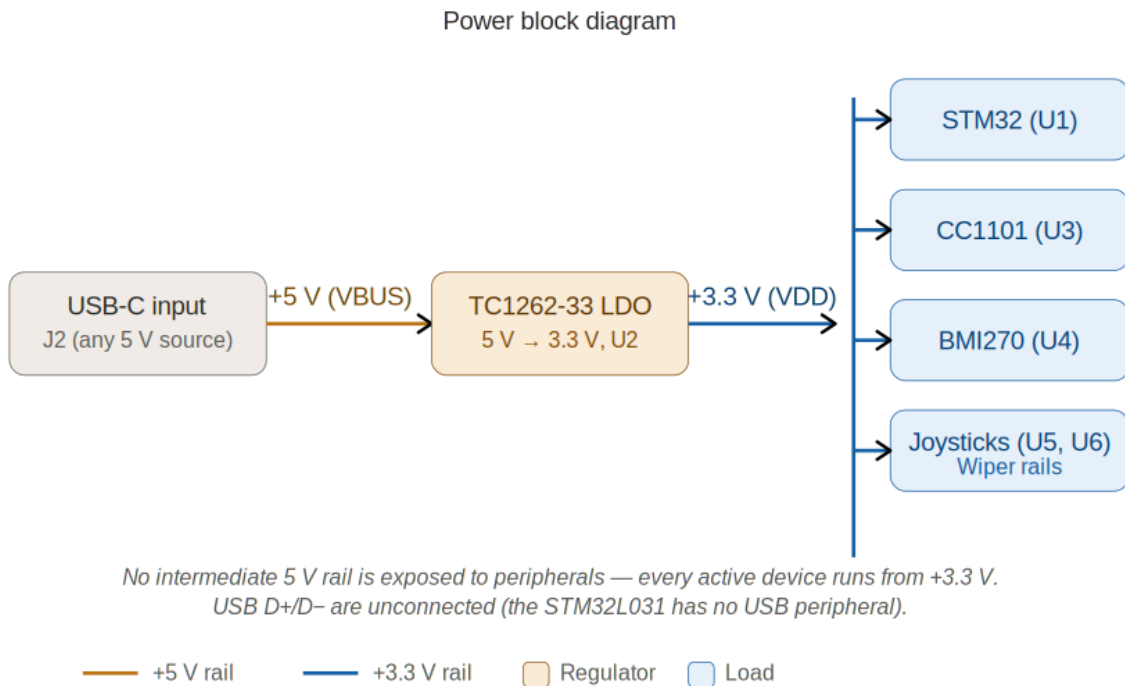
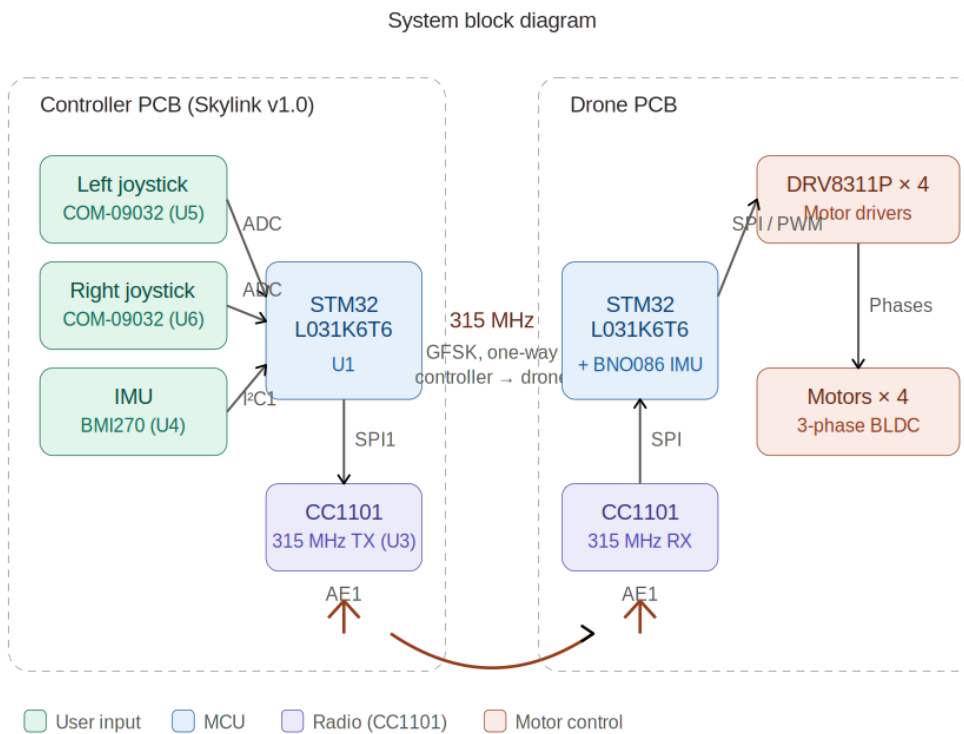
Overview

The overall goal of the project was to implement something that combined communication and motors in a fun and interesting way. After deliberation, the four of us decided to make a drone and then subsequently split into two groups. This group's job was to design and build a handheld controller to interface with the other groups' drone project. After brainstorming, our controller was designed to have analog joystick and tilt input and send the data over the 433MHz band (this was later tweaked to the 315MHz band).

Understanding the Drone

To understand the design behind the controller, you must first understand the design of the drone that it will be controlling. The main body of the drone is a PCB to limit the amount of mechanical engineering that needs to get involved. Uses the STM32L03KT6T6 as its MCU in order to control its motor control ICs (DRV8311P), BNO086 IMU, and CC1101 RF transceiver. The big overview is that the MCU first initializes all circuits, then enters its main loop where it receives controller data from the CC1101. Using this controller data, the MCU will send instructions to the DRV8311Ps to move the motors.

System Block Diagram



The controller has a deliberately simple two-rail power tree, summarized in Figure 2. The board accepts +5 V from a USB-C source (laptop, phone charger, or USB battery pack) and uses a Microchip TC1262-33 LDO to step it down to the +3.3 V rail that powers every active device on the board.

System Overview

The controller is organized into three subsystems: the MCU and its supporting power and programming circuitry, the sub-GHz RF transmit chain (CC1101 plus L/C network and antenna), and the user-input front end (two analog joysticks plus the BMI270 IMU). The STM32 sits at the center, reading user input on one side and driving the radio on the other.

The joystick and IMU sampling are decoupled from the radio. This keeps the main loop simple and predictable, and means the radio is the only thing that can stall the loop.

Board Operation

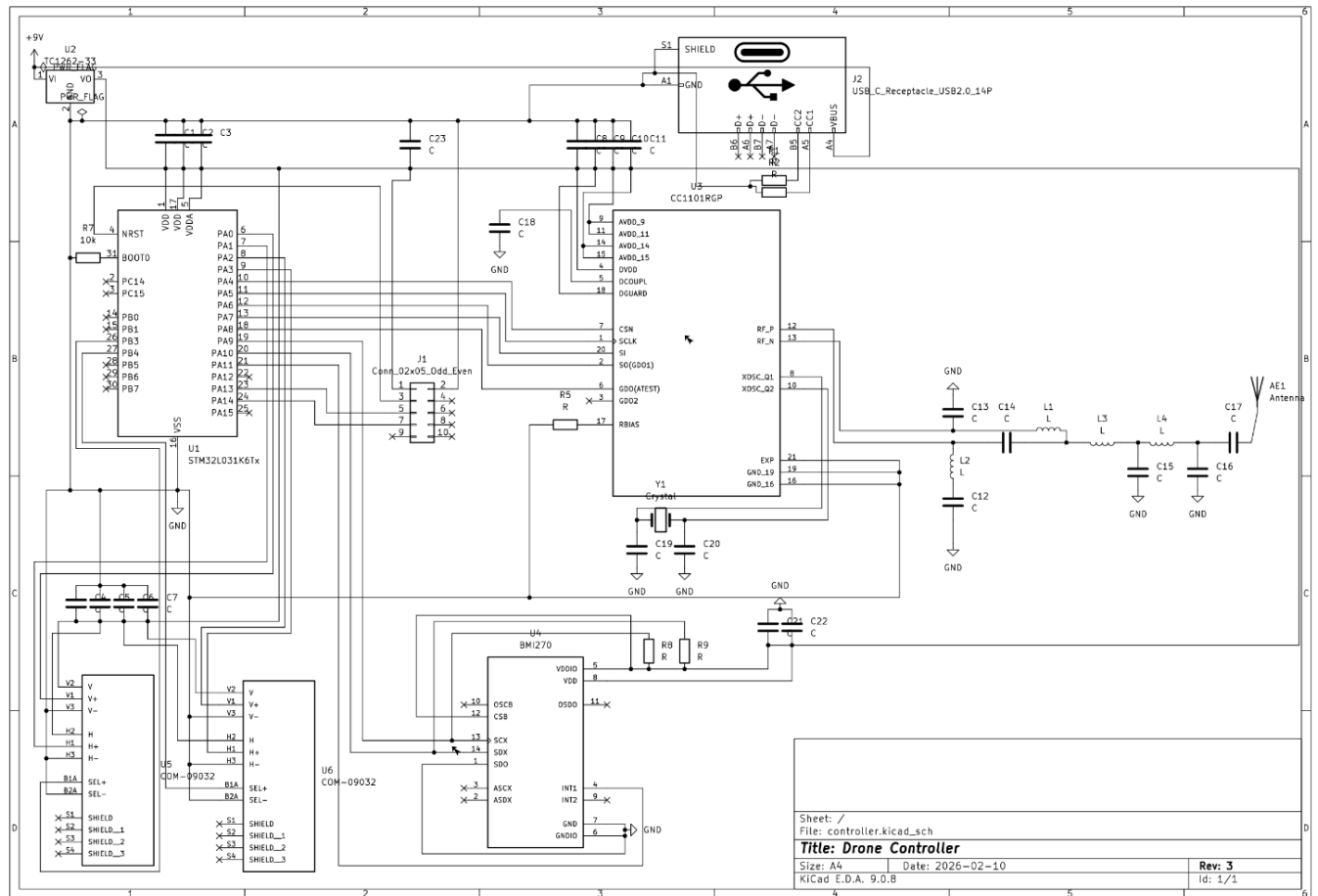
To use the controller, connect a USB-C cable from any 5 V source (laptop, phone charger, USB battery bank) to J2. Within power-up, the firmware finishes its init sequence and will start transmitting at the 50 Hz packet rate.

The two joysticks are positioned for thumb operation, in the standard “game controller” layout: left stick controls throttle (Y) and yaw (X), right stick controls pitch (Y) and roll (X). The pilot can additionally tilt the entire controller to bias the drone’s attitude.

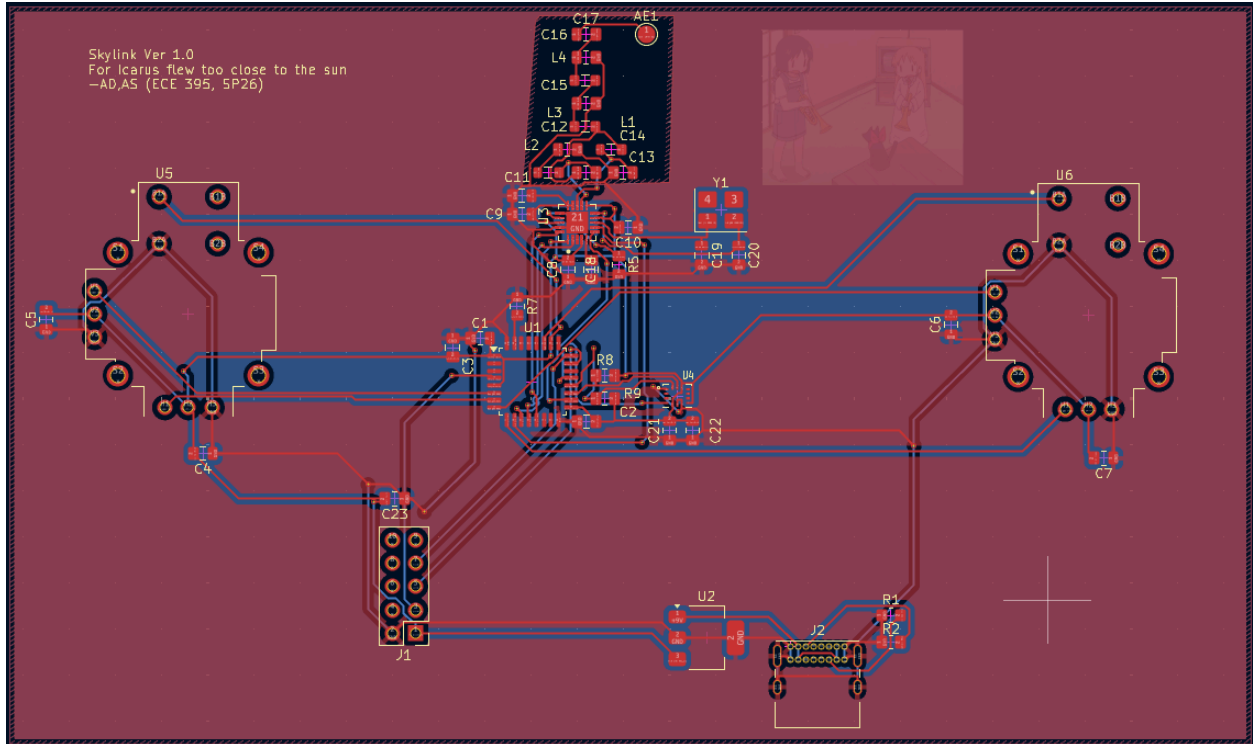
To program the board, connect an ST-LINK V2 or V3 to the 10-pin SWD header (J1) with the ST-LINK’s 3.3 V output enabled. The board will draw negligible current on the SWD-only power path, so a USB-C cable is not required for flashing.

Hardware Design

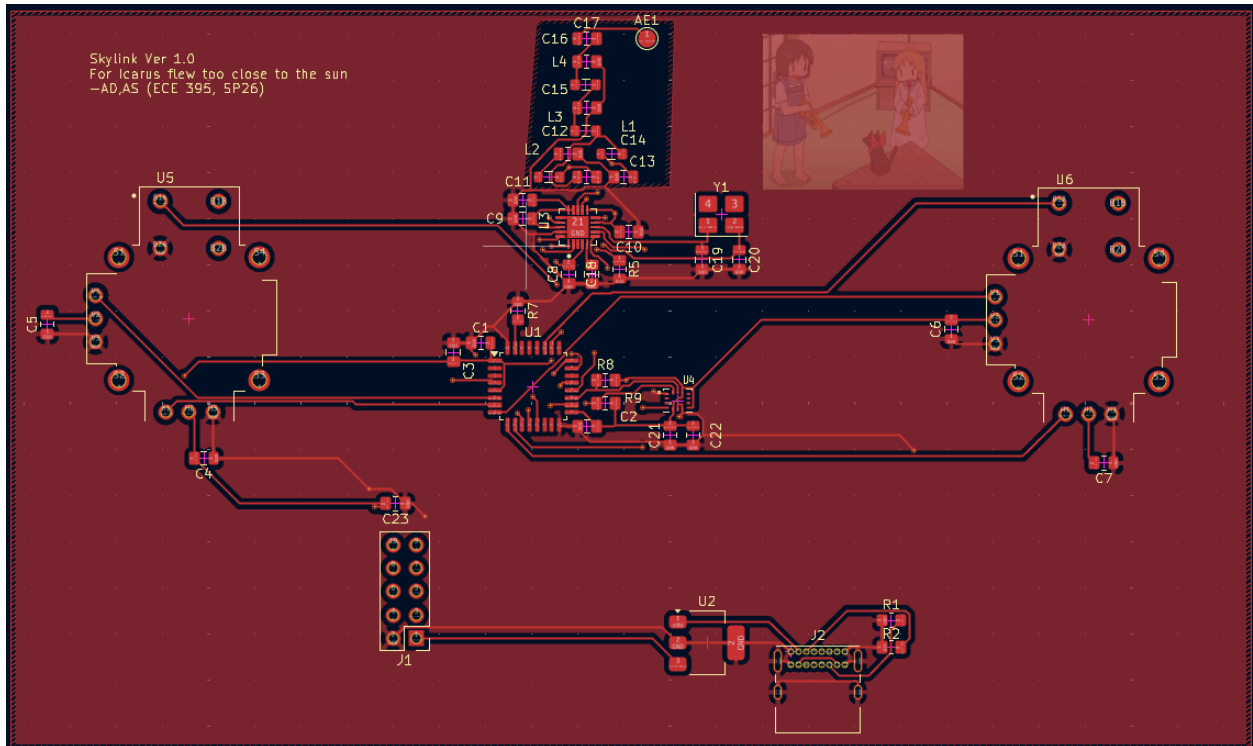
Overview



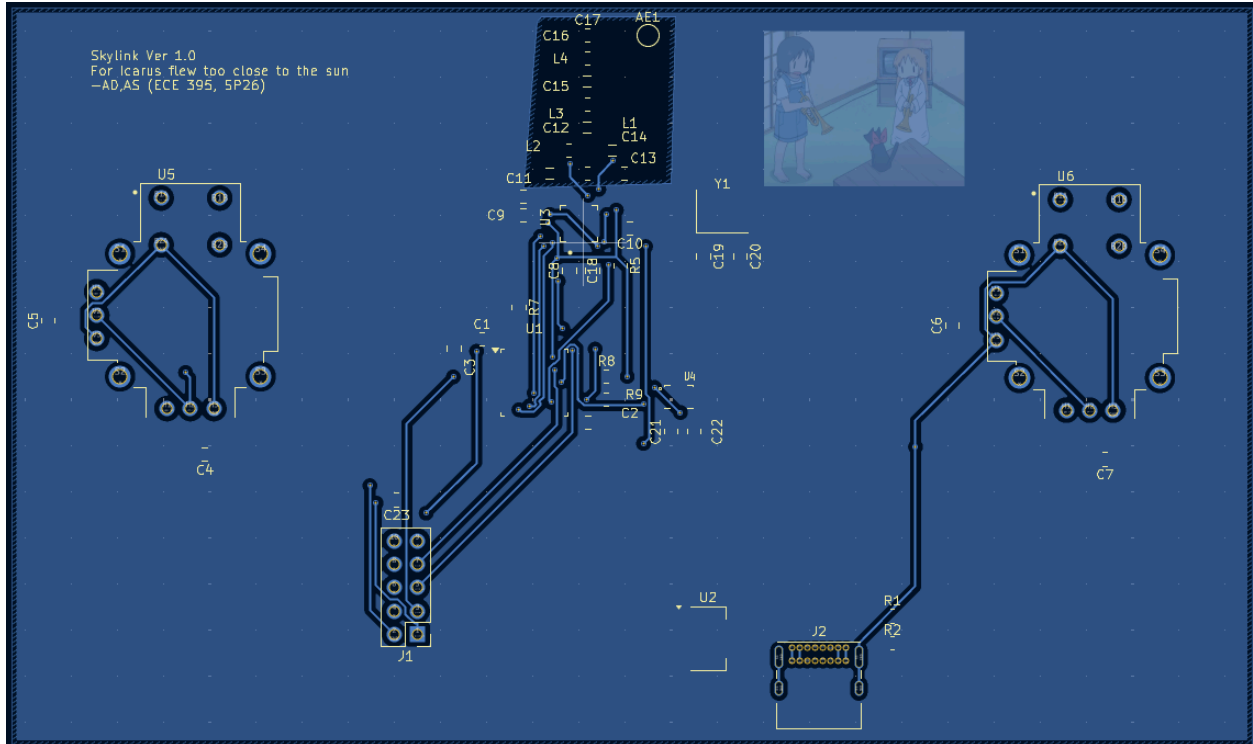
This is the final schematic for our design



This is the final PCB view of all layers



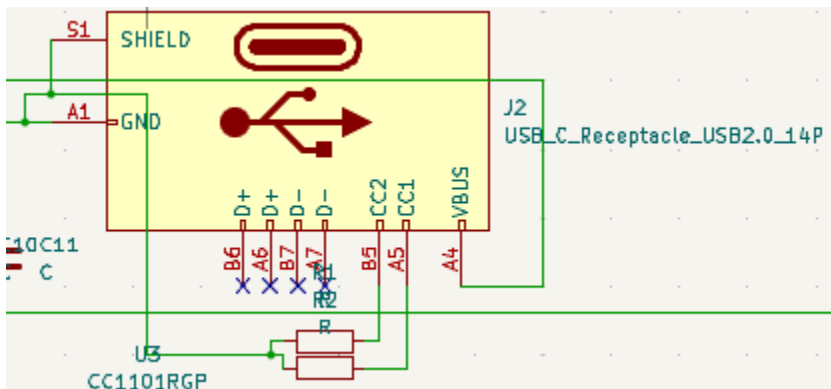
This is the PCB view of the just the Front Copper (and both silkscreens)



This is the PCB view of just the front copper (and both silkscreens)

Power/GND

The board accepts +5 V from USB-C (J2) and uses a low-dropout linear regulator (U2) to produce the +3.3 V VDD rail that powers the MCU, the CC1101, the BMI270, and the joystick potentiometer ladders. A series of bulk and high-frequency decoupling capacitors (C1–C3, C22–C23) are distributed around the regulator and around each active IC. Each VDD/VDDA pin on the STM32 has its own 100 nF cap placed within a few millimeters of the pin, with a 4.7 μ F bulk cap nearby.



Ground is a flood pour on the bottom layer with stitching vias around the RF section to keep the antenna return path short. The CC1101's ground pad is tied to the pour with a 2x2 via array.

Net	Voltage	Source	Used By
VBUS	+5 V	USB-C J2	Input to LDO U2
+3.3 V (VDD)	+3.3 V	U2 (LDO)	MCU, CC1101, BMI270, joystick wipers
GND	0 V	Ground pour	Common return

Microcontroller (STM32L031K6T6)

The STM32L031K6T6 is an ultra-low-power Cortex-M0+ part in a 32-pin LQFP with 32 KB of flash and 8 KB of SRAM. We picked it for three reasons: it is the same MCU the drone team selected and we used before in class (so we could share a CC1101 driver), its on-chip ADC is more than fast enough to sample two joysticks at our packet rate, and its SPI1 peripheral comfortably hits the 6.5 MHz the CC1101 tolerates on SCLK.

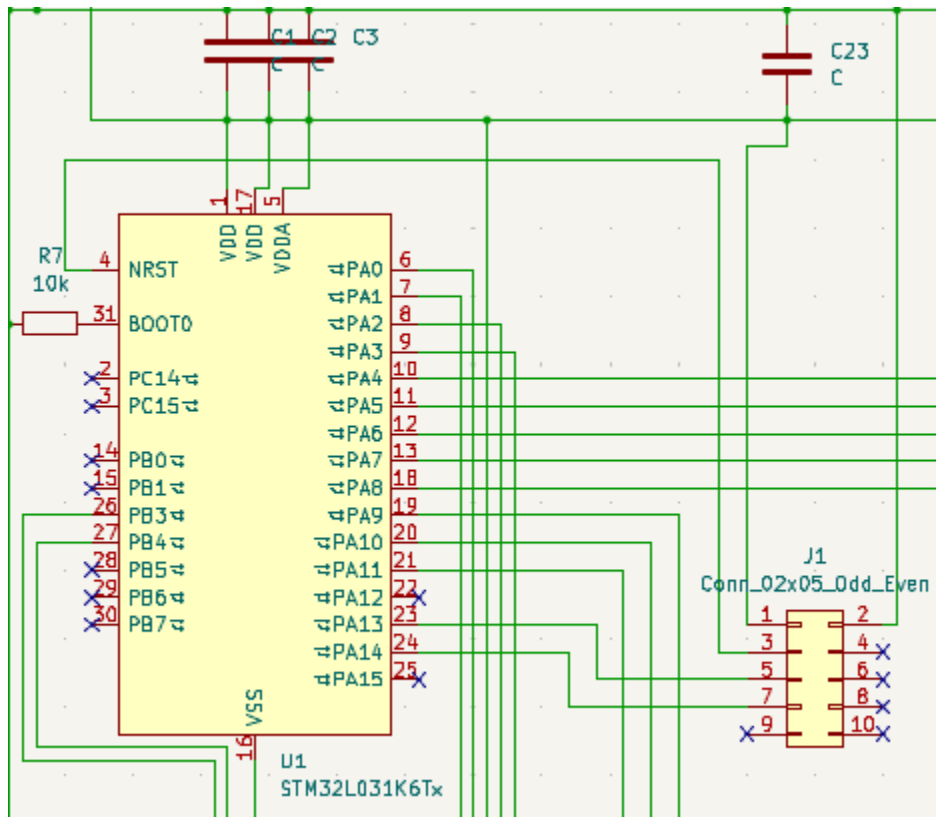


Table 1: STM32L031K6T6 pinout used on the controller (KiCad schematic).

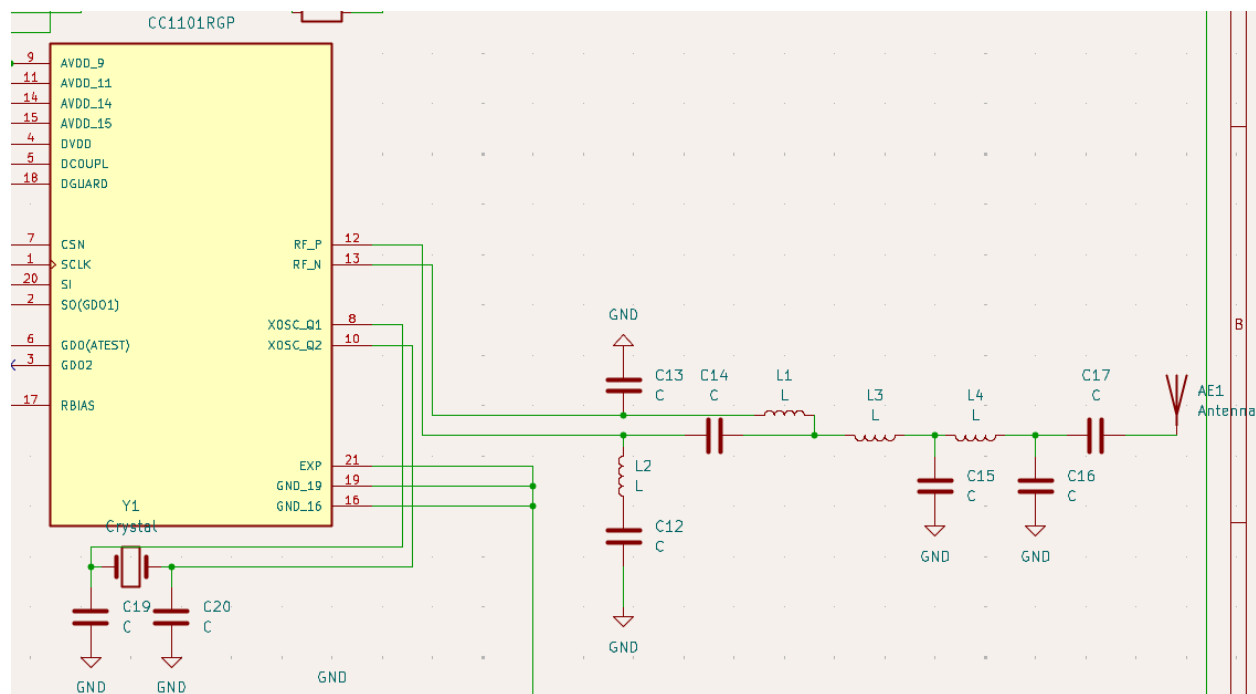
MCU Pin	Signal	Direction	Connected To	Notes
1, 17	VDD	Power	+3.3 V rail (U2 LDO output)	Two VDD pins; both decoupled
5	VDDA	Power	+3.3 V rail	Tied to VDD (no separate analog rail)

MCU Pin	Signal	Direction	Connected To	Notes
16, 32	VSS	GND	Ground pour	Both ground pins tied to GND
4	NRST	Input	J1 pin 3 (programming header)	Reset; driven by ST-LINK during programming
31	BOOT0	Input	R7 (10 kΩ to GND)	Held low; boots from user flash
6	PA0 / ADC_IN0	Analog in	U5 wiper V (left stick “vertical” axis)	12-bit ADC channel 0
7	PA1 / ADC_IN1	Analog in	U5 wiper H (left stick “horizontal” axis)	12-bit ADC channel 1
8	PA2 / ADC_IN2	Analog in	U6 wiper V (right stick V axis)	12-bit ADC channel 2
9	PA3 / ADC_IN3	Analog in	U6 wiper H (right stick H axis)	12-bit ADC channel 3
10	PA4 / SPI1_NSS	Output	CC1101 CSN	Software-controlled chip select
11	PA5 / SPI1_SCK	Output	CC1101 SCLK	SPI clock
12	PA6 / SPI1_MISO	Input	CC1101 SO/GDO1	SPI data in from radio
13	PA7 / SPI1_MOSI	Output	CC1101 SI	SPI data out to radio
18	PA8	Input (EXTI)	CC1101 GDO0	Configured as end-of-packet interrupt
19	PA9 / I2C1_SCL	OD output	BMI270 SCX (I ² C clock)	10 kΩ pull-up (R8) to VDD
20	PA10 / I2C1_SDA	OD bidir	BMI270 SDX (I ² C data)	10 kΩ pull-up (R9) to VDD
21	PA11	Input (EXTI)	BMI270 INT1	Data-ready interrupt
23	PA13 / SWDIO	Bidir	J1 pin 5 (programming header)	ST-LINK SWD data
24	PA14 / SWCLK	Input	J1 pin 7 (programming header)	ST-LINK SWD clock

MCU Pin	Signal	Direction	Connected To	Notes
26	PB3	Input	U5 SEL+ (left stick button)	Internal pull-up enabled in firmware
27	PB4	Input	U6 SEL+ (right stick button)	Internal pull-up enabled in firmware
2, 3, 14, 15, 22, 25, 28, 29, 30	PC14, PC15, PB0, PB1, PA12, PA15, PB5, PB6, PB7	—	No-connect	Available for future expansion

Communications

The CC1101 is a single-chip sub-GHz transceiver from TI. The MCU drives it over SPI1, with two additional digital lines (CSn and GDO0). GDO0 is configured to assert at end-of-packet, which we wire to an STM32 EXTI line so the firmware can sleep between TXes.



Originally the link was specified at 433.92 MHz, and the matching network around the CC1101 (L1–L4 and C13–C16) was laid out using the values from TI’s 433 MHz reference design: $L1/L2/L4 = 27 \text{ nH}$, $L3 = 22 \text{ nH}$, $C13/C14 = 3.9 \text{ pF}$, $C15 = 8.2 \text{ pF}$, $C16 = 5.6 \text{ pF}$. When we populated and tested the board, the return loss at 433 MHz was poor and we could not close the link reliably.

Rather than respin the PCB, we tried reworked the matching network in place to values that, combined with the quarter-wave whip antenna AE1 we had on hand, produced a noticeably cleaner match at 315 MHz: $L1/L2/L4 = 33 \text{ nH}$, $L3 = 18 \text{ nH}$, $C13/C14 = 6.8 \text{ pF}$, $C15 = 12 \text{ pF}$, $C16 = 6.8 \text{ pF}$. C12 and C17 (220 pF) act as RF DC-blocking caps on either side of the network and were left unchanged. We then reprogrammed the CC1101’s base-frequency registers

(FREQ2/FREQ1/FREQ0) for 315 MHz and verified link operation at that band. The 315 MHz ISM allocation in the US permits short-range device operation under FCC Part 15, which is appropriate for a tethered demo project.

Y1 is the 26.0 MHz SMD-3225 crystal used by the CC1101 PLL, with C19 and C20 (both 27 pF) as load caps. R5 (56 k Ω , RBIAS) sets the internal current reference.

Table 3 lists the as-populated RF front-end BOM values, including the deltas from the original 433 MHz design.

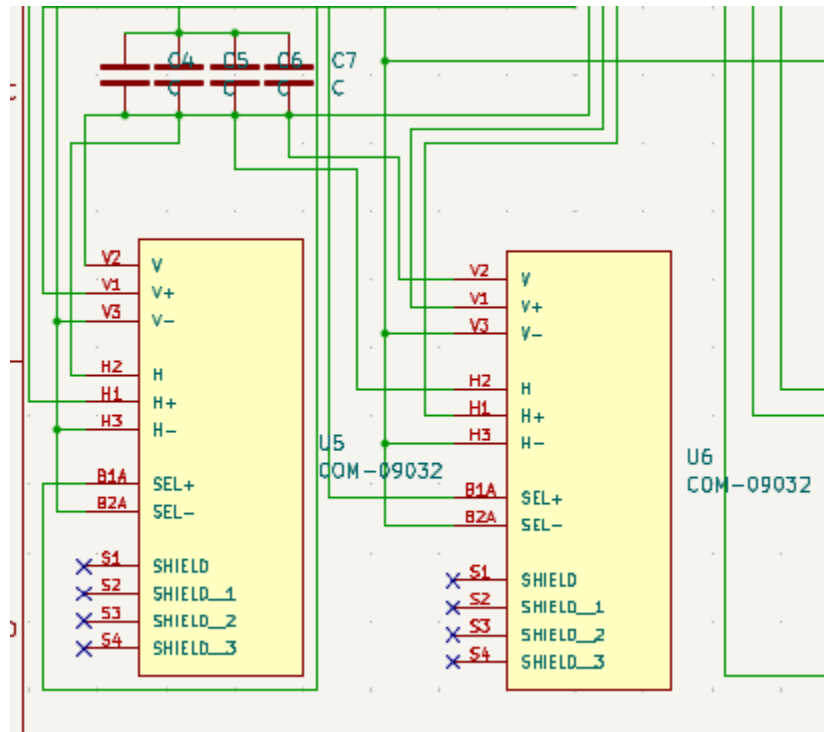
Ref	Function	Schematic Value (433 MHz)	As-Populated Value (315 MHz)
L1	Matching network	27 nH	33 nH
L2	Matching network	27 nH	33 nH
L3	Matching network	22 nH	18 nH
L4	Matching network	27 nH	33 nH
C13	Matching network	3.9 pF	6.8 pF
C14	Matching network	3.9 pF	6.8 pF
C15	Matching network	8.2 pF	12 pF
C16	Matching network	5.6 pF	6.8 pF
C12	RF DC block	220 pF	220 pF (unchanged)
C17	RF DC block	220 pF	220 pF (unchanged)
C19	Crystal load	27 pF	27 pF
C20	Crystal load	27 pF	27 pF
Y1	PLL crystal	26.0 MHz SMD-3225	26.0 MHz SMD-3225
R5	RBIAS	56 k Ω	56 k Ω

CC1101 Pin	Signal	MCU Pin	Notes
CSN (pin 7)	SPI chip select	PA4	Active-low, software-controlled
SCLK (pin 1)	SPI clock	PA5	Up to 6.5 MHz
SI (pin 20)	SPI data in	PA7	MOSI
SO / GDO1 (pin 2)	SPI data out	PA6	MISO
GDO0 (pin 6)	Status output	PA8 (EXTI)	Configured as end-of-packet
GDO2 (pin 3)	Status output	—	Not connected
XOSC_Q1/Q2 (8, 10)	Crystal	Y1 (26 MHz)	Load caps C19, C20 (27 pF)
RBIAS (pin 17)	Bias resistor	—	R5 = 56 k Ω to GND

CC1101 Pin	Signal	MCU Pin	Notes
RF_P / RF_N (12, 13)	Differential RF	—	Matching network L1–L4 / C13–C16 to AE1

Joystick Input (COM-09032 × 2)

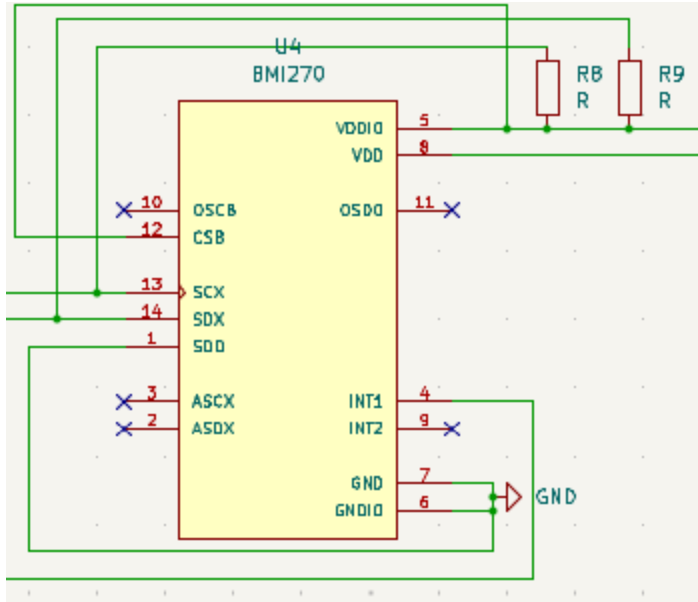
Each COM-09032 module from SparkFun is a Nintendo-style thumb-joystick: two orthogonal potentiometers (one for X, one for Y) to VDD/GND, plus a pushbutton that closes when the user clicks the stick straight down.



Both joysticks share VDD and GND with the rest of the board. Their four wipers run to PA0–PA3, which the STM32's on-chip 12-bit SAR ADC samples in scan mode. The two pushbutton outputs go to PA4 and PB6 and read as active-low GPIOs.

Gyroscope / IMU Input (BMI270)

The BMI270 is a Bosch 6-axis IMU (3-axis accelerometer + 3-axis gyroscope) with an internal DSP. On the controller we use it strictly for tilt input: the firmware reads the accelerometer at 100 Hz over I²C, computes roll and pitch from the gravity vector, and folds those two angles into the outgoing packet alongside the joystick values. The pilot can use this to bias the drone's attitude by tilting the controller itself, which is more intuitive than fine joystick movement.



The IMU sits on I2C1 (PB3 SCL, PB4 SDA) with 10 kΩ pull-ups to VDD (R8, R9). If we re-spin the board we would drop these to 2.2 kΩ. ASCX/ASDX (the auxiliary I²C interface to an external magnetometer) are pulled to VDD because we do not populate one.

BMI270 Pin	Signal	MCU Pin	Notes
SCK / SCX	I ² C clock	PB3	10 kΩ pull-up (R8)
SDI / SDX	I ² C data	PB4	10 kΩ pull-up (R9)
INT1	Data-ready	PB5 (EXTI)	Configured push-pull active-high
INT2	(unused)	—	Routed but unused
ASCX/ASDX	Aux I ² C	—	Pulled to VDD, no aux device
VDDIO/VDD	Supply	+3.3 V	Decoupling: C22
GND/GNDIO	Ground	GND	—

Headers and Connectors

Two off-board connectors and one programming header are exposed.

Reference	Type	Purpose
J1	10-pin 2x5 0.05" header	ST-LINK SWD programming + ground / VDD test points
J2	USB-C receptacle (USB 2.0 14-pin)	+5 V power input

Reference	Type	Purpose
AE1	Quarter-wave whip / wire antenna	315 MHz RF output from CC1101 matching network

Programming header pinout (J1, viewed from the top of the PCB):

Pin	Signal
1	+3.3 V (VDD)
2	SWDIO (PA13)
3	GND
4	SWCLK (PA14)
5	GND
6	NRST
7	GND
8	(NC / test point)
9	+5 V (VBUS)
10	GND

Firmware Design

Overview

The firmware running on the STM32 is responsible for initializing/configuring the CC1101, BMI270, reading joystick/sensor inputs, stuffing the data into a packet, and sending the data with the antenna.

STM Configuration

ADC_IN0: This ADC is used to read in analog voltage from the left joystick horizontal direction and turn it into digital signals to then send in a packet

ADC_IN1: This ADC is used to read in analog voltage from the left joystick vertical direction and turn it into digital signals to then send in a packet

ADC_IN2: This ADC is used to read in analog voltage from the right joystick horizontal direction and turn it into digital signals to then send in a packet

ADC_IN3: This ADC is used to read in analog voltage from the right joystick vertical direction and turn it into digital signals to then send in a packet

SPI1_MISO: This is used to receive data from the CC1101; not used on the controller side except for debugging but is used on the drone side to receive the packet information

SPI1_MOSI: This is used to send data to the CC1101; usually sends packet information

SPI1_SCK: This sends a clock signal used to synchronize with the CC1101

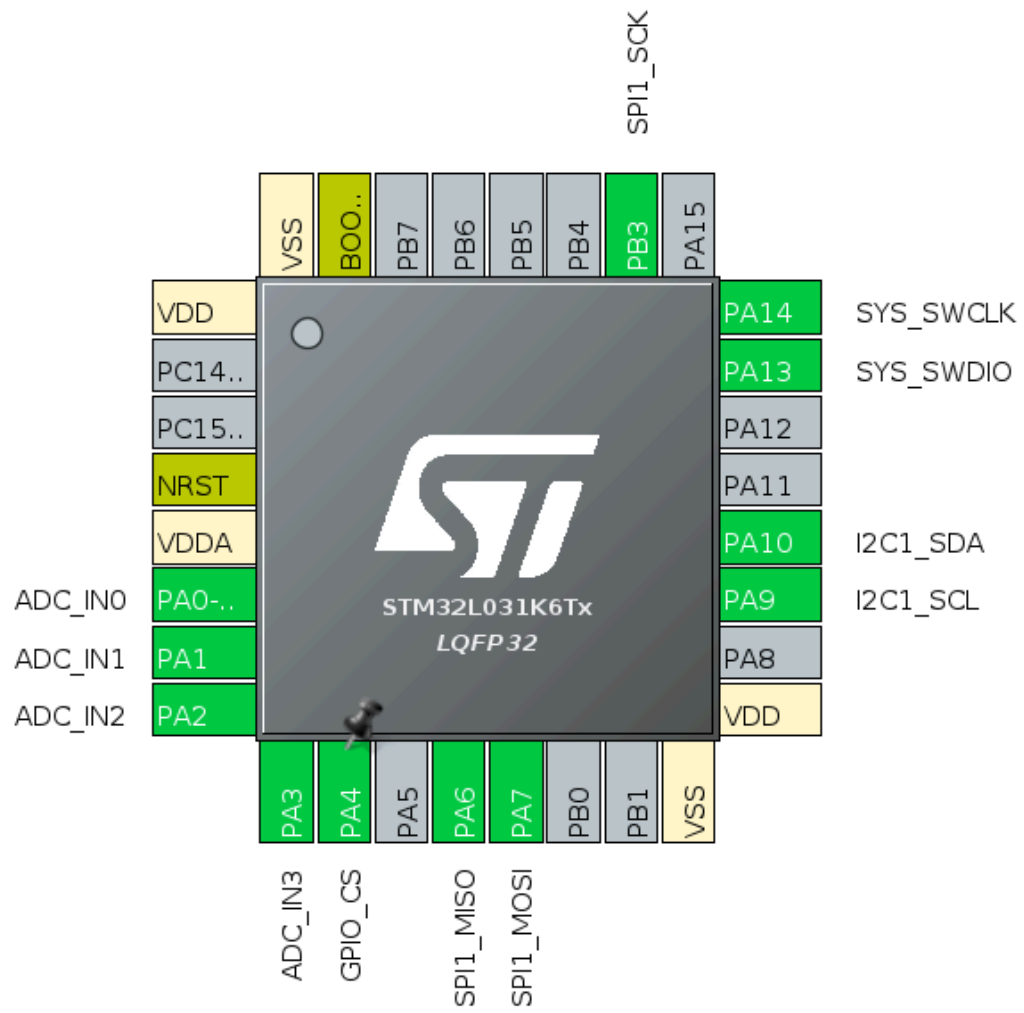
GPIO_CS: This pin is used to tell the CC1101 when it is being communicated with

I2C1_SDA: This is the data signal used to get tilt data from the IMU

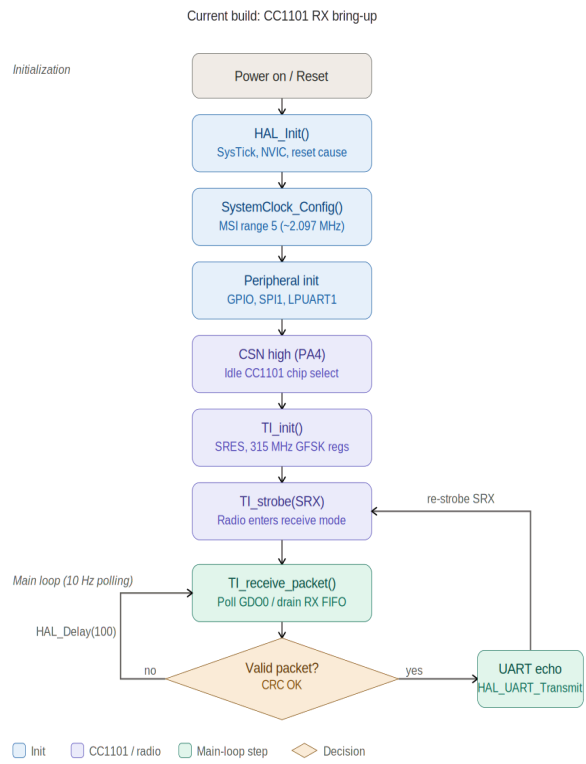
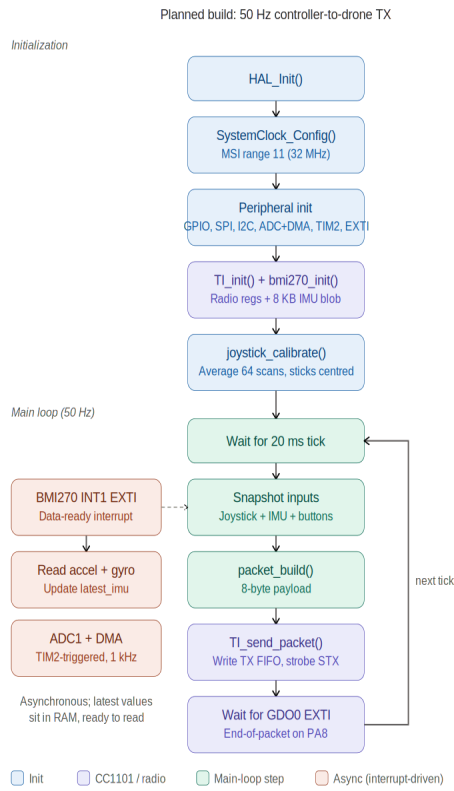
I2C1_SCL: This is the clock signal used to synchronize with the IMU

SYS_SWCLK: This was used as a reference clock signal

SYS_SWDIO: This was used to debug using the ST-Link



Flowchart



Initialization

```

HAL_Init();

SystemClock_Config();

MX_GPIO_Init();
MX_SPI1_Init();
MX_ADC_Init();
MX_I2C1_Init();

HAL_GPIO_WritePin(CC1101_CS_PORT, CC1101_CS_PIN, GPIO_PIN_SET);

TI_init(&hspi1, CC1101_CS_PORT, CC1101_CS_PIN);
  
```

This code initializes the GPIO, SPIO, ADC, and I2C and sets up the CC1101.

Main Loop

```
uint8_t txBuffer[(ADC_VAL_CNT + 2*2)*2];
uint16_t adc_val[ADC_VAL_CNT + 2*2];
// int count = 0;
while (1)
{
    // ----- SEND -----

    Read_ADC_4CH(adc_val); //read joystick data
    BMI270_ReadAccel(&hi2c1, &bmi); //get bmi data
    adc_val[ADC_VAL_CNT] = bmi.roll; //get the roll
    adc_val[ADC_VAL_CNT + 2] = bmi.pitch; //get the pitch
    Pack_Buffer(txBuffer, adc_val, (ADC_VAL_CNT + 2*2)*2); //pack it
    TI_send_packet(txBuffer, (ADC_VAL_CNT + 2*2)*2); //send it
    HAL_Delay(10); //small delay

}
```

This code first creates a buffer to be used throughout the main loop. Then it reads in the analog joystick data using `Read_ADC_4Ch` and puts it into the buffer. It then reads the acceleration data and puts that in the same buffer. Then we pack the buffer into a different one. This is done more to standardize how all of the data is stored and also so that we are sending bytes at a time. Then we send the packet and introduce a small delay before we loop again.

Helper Functions

```
void Read_ADC_4CH(uint16_t *adc_val)
{
    HAL_ADC_Start(&hadc);

    for (int i = 0; i < 4; i++)
    {
        if (HAL_ADC_PollForConversion(&hadc, HAL_MAX_DELAY) == HAL_OK)
        {
            adc_val[i] = HAL_ADC_GetValue(&hadc);
        }
    }

    HAL_ADC_Stop(&hadc);
}

void Pack_Buffer(uint8_t* buffer, uint16_t* values, int valCount){
    for(int i = 0; i < valCount; i++){
        buffer[i*2] = (values[i] >> 8) & 0xFF; //MSB
        buffer[i*2+1] = (values[i]) & 0xFF;    //LSB
    }
}
```

The top function reads the data from the joysticks and puts it in a buffer and the bottom function takes the joystick and IMU values, packs it, and returns it for sending.

```
void TI_send_packet(BYTE* txBuffer, UINT8 size)
{
    BYTE status;

    TI_strobe(CCxxx0_SIDLE); //ïäöâäïäèì ìäâì ä IDLE

    TI_write_reg(CCxxx0_TXFIFO, size);

    status = TI_read_status(CCxxx0_TXBYTES);

    TI_write_burst_reg(CCxxx0_TXFIFO, txBuffer, size);
    status = TI_read_status(CCxxx0_TXBYTES);

    TI_strobe(CCxxx0_STX);
}
```

This function is called by main and is used to send the packets with the CC1101.

Conclusion

Issues

Honestly, the biggest issue we ran into was pacing. We spent too long on the schematic and PCB iteration in the first half of the semester and did not start firmware bring-up in parallel on a development board as quickly. By the time the PCB came back from fabrication we were trying to debug the CC1101, the joystick ADC chain, and the BMI270 configuration upload all at once, which compounded every issue we hit.

Our board was also programmable and tested on a development board, but when soldering on the PCB components, we had our PCB short circuit (connectivity in one resistor area which led to our board operating at a third of the original voltage) and were unable to respin another board in time.

What could be done in the future?

- Re-spin the RF matching network for 433 MHz with the L/C values pulled directly from the TI CC1101 reference design rather than improvised from what was in the lab.
- Add a small OLED to the controller so the pilot can see live joystick / IMU / link-quality values without a USB serial cable.
- Add a LiPo charger (MCP73831 or similar) and a battery so the controller is actually wireless on the controller side as well.
- Implement bidirectional link with telemetry on the return path.
- Implement more efficient communication with error correction/detection

Tips for future projects regarding drone controllers

- Starting as early as possible. We spent so much time trying to make the right design for our PCB that we delayed ordering one until the last submission day. This basically had us stuck to that one PCB and if it didn't work, we couldn't re-order another one.
- Doing work in progress on a dev board. While we were waiting for our PCB, we should have spent more time playing with the dev board and the firmware and ensuring that our code and project at least work on a breadboard.
- Add test points and LED indicators in your board. Every SPI line, every I²C line, every supply rail, and the CC1101 GDO0 pin should have a labeled test point.
- Talk with Michael more.

Final Remarks

In its previous state, the controller successfully sampled both joysticks and the IMU, packed their values into an 8-byte payload, and transmitted them over a 315 MHz CC1101 link to the drone. The two halves were integrated late enough in the semester that we did not get a fully airborne demo, but the wireless link itself is functional and the controller PCB is a usable platform for future ECE 395 teams to build on. Most of what slowed us down was schedule discipline rather than any single technical problem, and we hope the advice above helps the next group avoid the same trap.